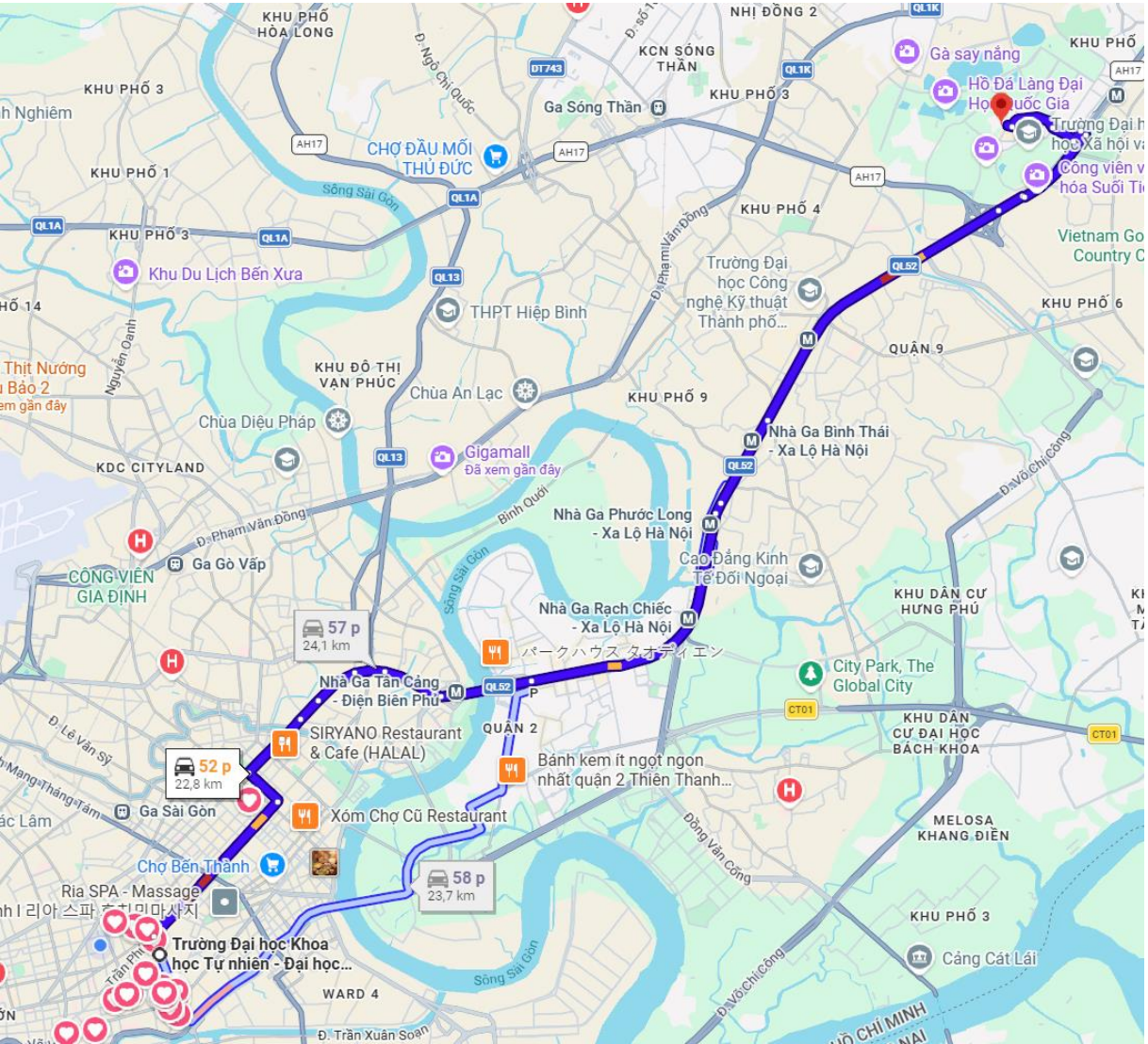
CS163 – Shortest Path Algorithm

Members:
25125045: Tran Nhu Khai
25125074: Nguyen Anh Kiet
25125083: Nguyen Dinh Minh Huy
25125014: Nguyen Duc Huy

@2023, University of Science HCMC - Advanced Program in Computer Science

Introduction

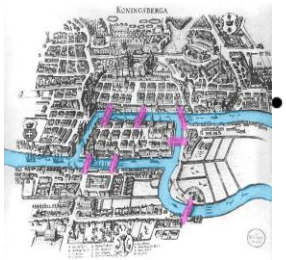


- Which way is the fastest?
- How do computers navigate thousands of turns?

History of shortest path

1736

Leonhard Euler solved the bridge problem, giving birth to **Graph Theory**.



1956-1958

Bellman-Ford: Finds **shortest paths** even with negative edge weights.

1959

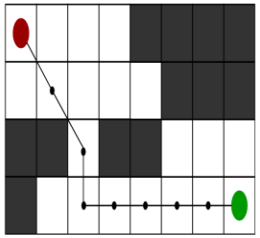
Dijkstra's Algorithm: The legendary algorithm for finding the absolute **shortest path** from A to B.

1962

Floyd-Warshall: Computes **shortest paths** between all pairs of nodes simultaneously.

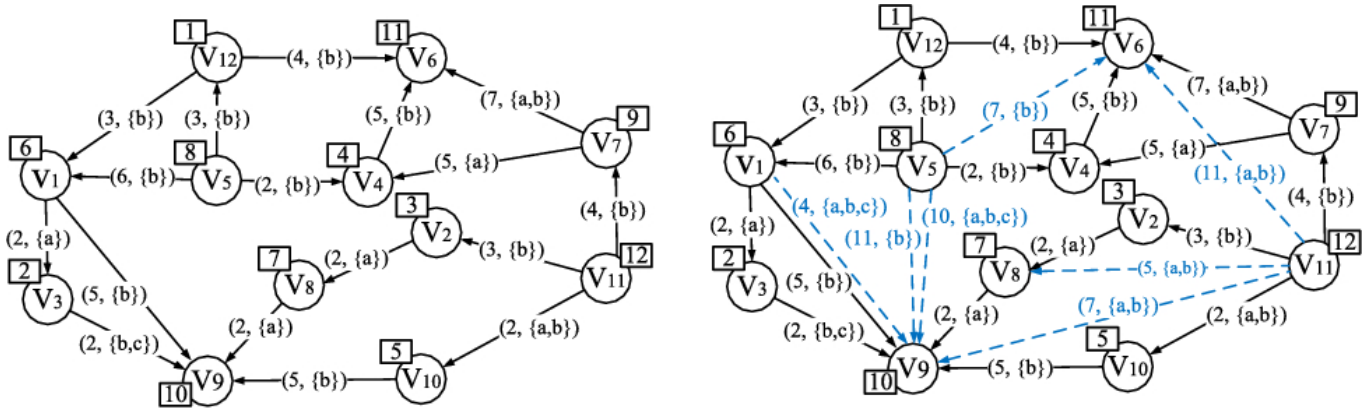
1968

A* algorithm: A faster **shortest path** finder using heuristics to guide the AI search.



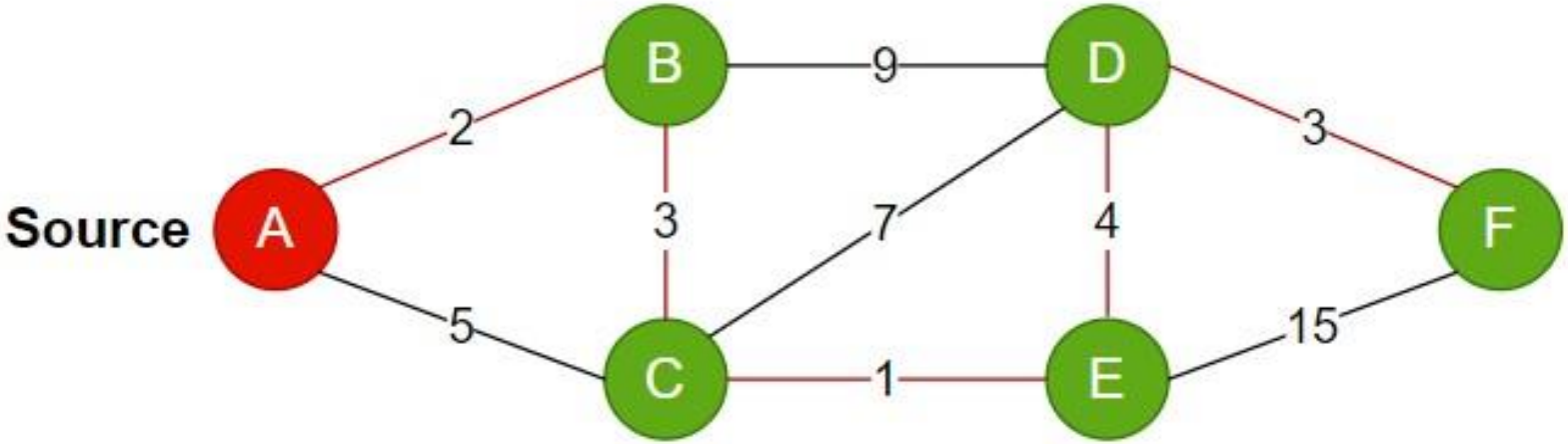
In today's world

With the launch of Google Maps, representing the real world requires graphs with billions of nodes. Traditional shortest-path algorithms became fundamentally too slow for real-time user queries. Researchers introduced advanced techniques like **Contraction Hierarchies (2008)** and Hub Labeling to pre-compute the map data. Thanks to this, modern servers can calculate the shortest path across an entire country in mere milliseconds.



(a) A road network G (b) CHLR on G

Single Source Shortest Path (SSSP)

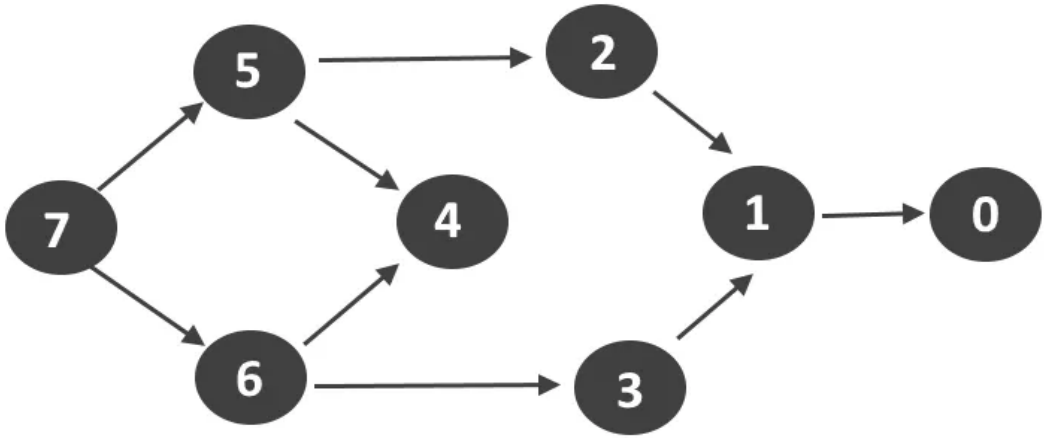


What is the shortest path from one starting node to all other nodes?

Shortest Path in DAGs

Directed Acyclic Graph (DAG): *A directed graph which forms no cycles or loops*

Topological Order: *A linear ordering of its vertices such that for every directed edge from vertex u to vertex v (u -> v), vertex u comes before vertex v in the ordering*



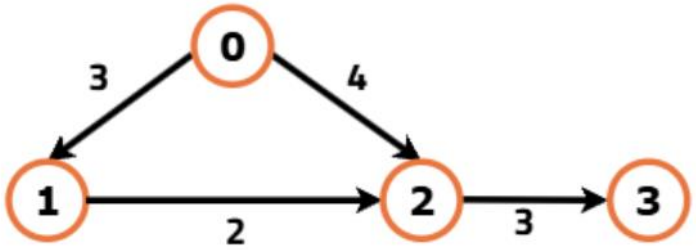
Topological Sort : 7 6 5 4 3 2 1 0

Shortest Path in DAGs

The Single Source Shortest Path (SSSP) problem can be solved efficiently on a DAG. This is due to the fact that the nodes can be ordered in a topological ordering via topsort and processed sequentially

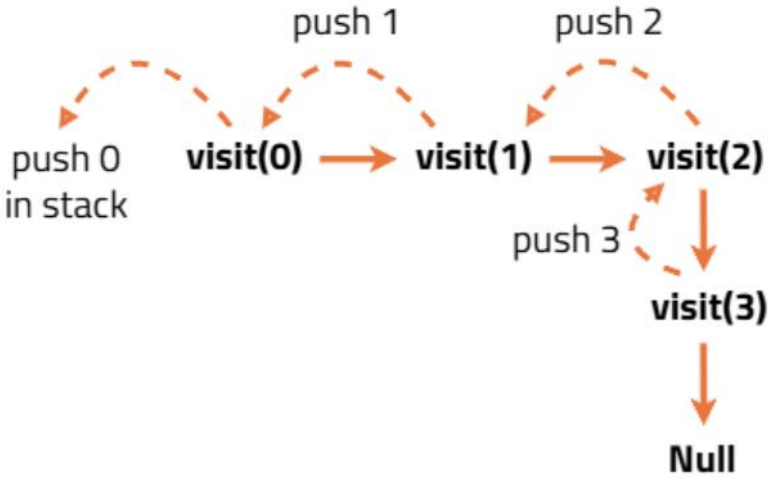
1. Select the first coming vertex from the topological ordering
2. Explore all its neighbours
3. If finding a shorter path, relax the edge.
4. Reprocess until finishing

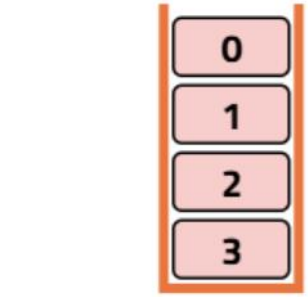
Shortest Path in DAGs



0: (1, 3), (2, 4)
1: (2, 2)
2: (3, 3)
3: _

create
adjacncy list



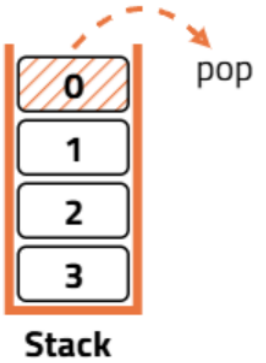


Stack

0	∞	∞	∞

dist

Shortest Path in DAGs

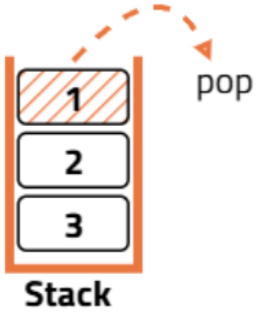


dist[1] = min[∞ , 0 + 1] = 1

dist[2] = min[∞ , 0 + 4] = 4

0	1	4	∞

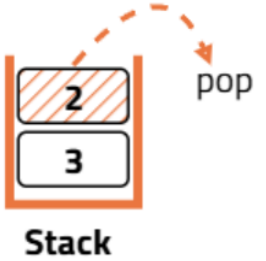
dist



dist[2] = min[4 , 1 + 2] = 3

0	1	3	∞

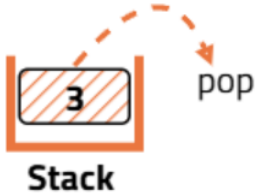
dist



dist[3] = min[∞ , 3 + 3] = 6

0	2	3	6

dist



No outgoing edges --> no change

final shotest dist =

0	1	3	6

Shortest Path in DAGs

Topological Sort Pseudocode

```
function topologicalSort(graph):
    visited = [false, false, … , false] # size V
    order = empty stack
    for i in range(0, V):
        if not visited[i]:
            dfs(graph, i , visited, order)
    return order

function dfs(graph, index, visited, order):
    visited[index] = true
    for edge in graph[index]:
        if not visited[edge.to]:
            dfs(graph, edge.to, visited, order)
    order.push(index)
```

Shortest Path in DAGs

Shortest Path Pseudocode

```
function dagShortestPath(graph, start):
    distances = [∞, ∞, … , ∞] # size V
    distances[start] = 0
    topoOrder = topologicalSort(graph)
    while topoOrder is not empty:
        index = topoOrder.pop()
        if distances[index] == ∞: continue
            for edge in graph[index]:
                newDist = distances[index] + edge.cost
                if newDist < distances[edge.to]:
                    distances[edge.to] = newDist
    return distances
```

Shortest Path in DAGs

Complexity Analysis

Time Complexity

We are traversing every vertices and edges. Therefore, the time complexity is:

$$O(V + E)$$

Space Complexity

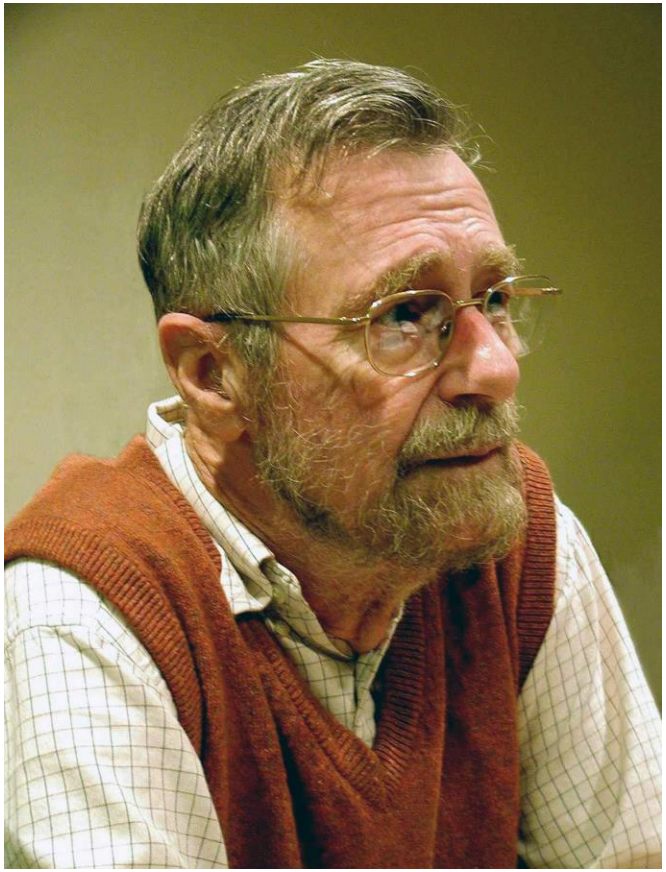
We are storing the topological ordering and the distances array. Therefore, the space complexity is:

$$O(V)$$

Dijkstra's Algorithm

Given a graph of non-negative weight edges. We can greedily assume that adding edges can only increase the total path cost.

Therefore, we come up with the **Greedy** strategy: continually select the unvisited node with the smallest known distance using a min-heap assuming it cannot be improved anymore, and then relaxes its outgoing edges.



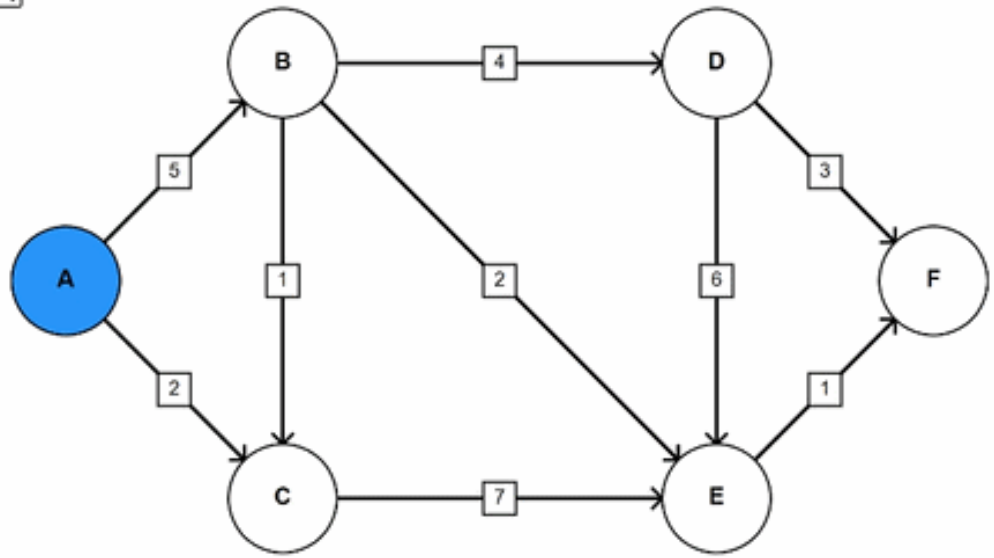
Edsger W. Dijkstra

(1930 – 2002)

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

Open

A	B	C	D	E

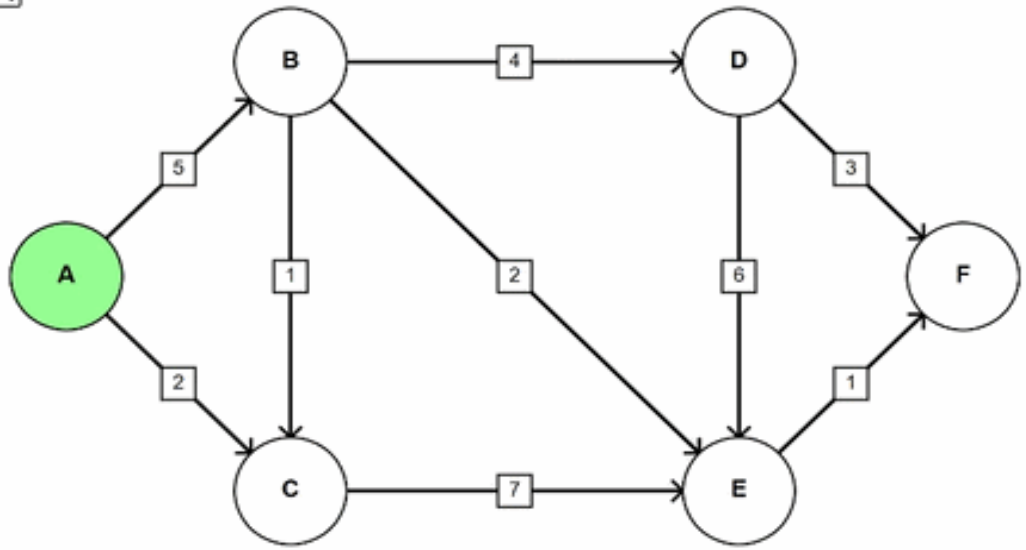
Closed

A						

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	5	A
C	2	A
D	∞	
E	∞	
F	∞	

Open

B	C	D	E

Closed

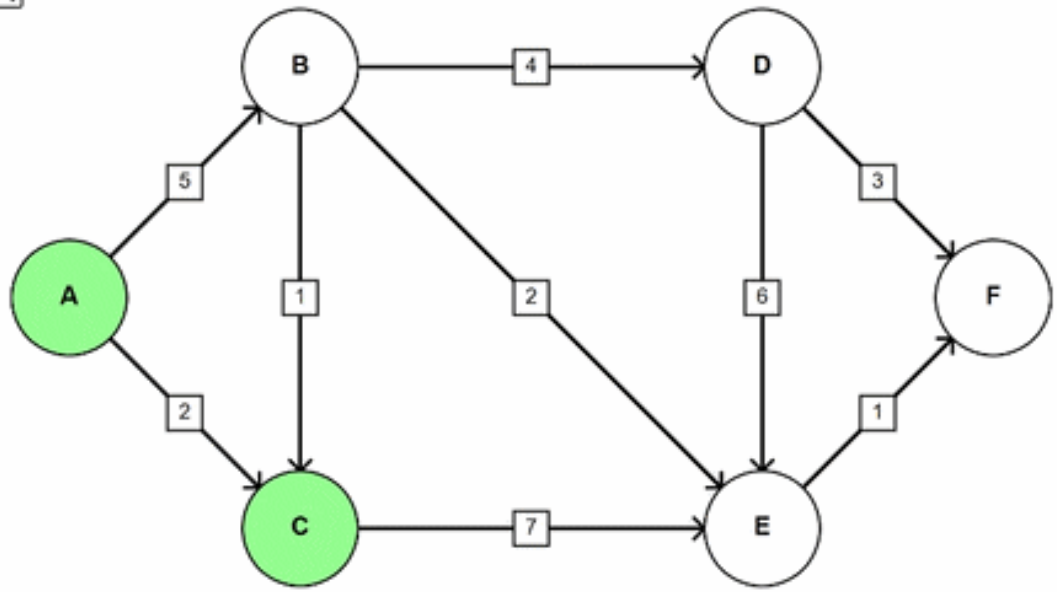
A

C	B					

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	5	A
C	2	A
D	∞	
E	9	C
F	∞	

Open

B	D	E

Closed

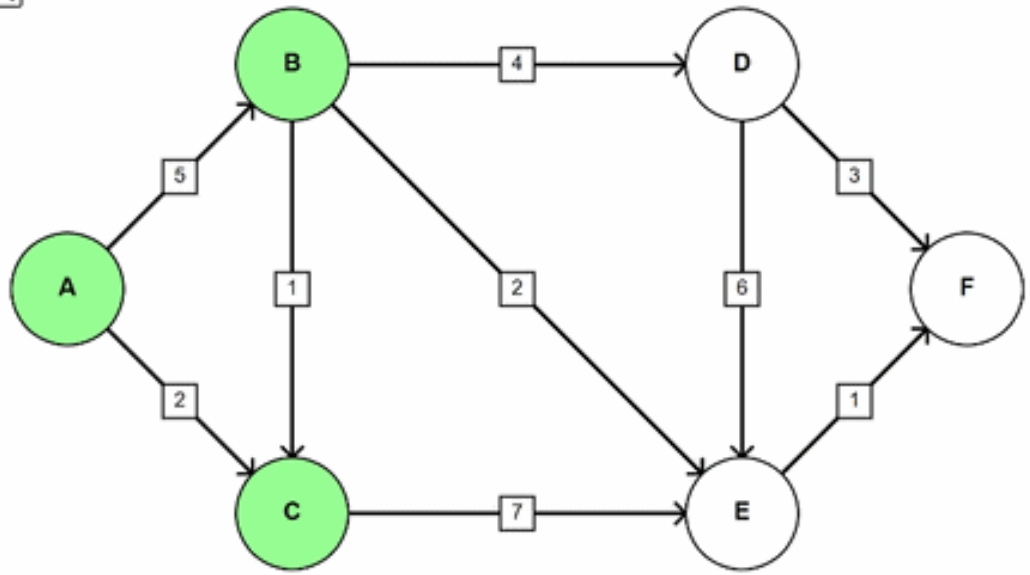
A	C

B	E	D				

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	5	A
C	2	A
D	9	B
E	7	B
F	∞	

Open

D	E

Closed

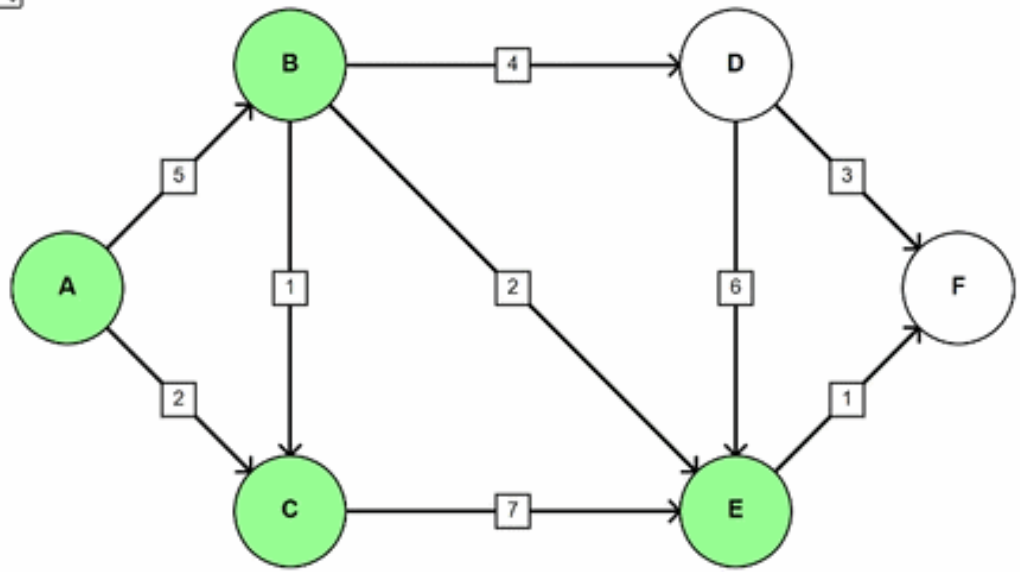
A	C	B

E	D					

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	5	A
C	2	A
D	9	B
E	7	B
F	8	E

Open

D

Closed

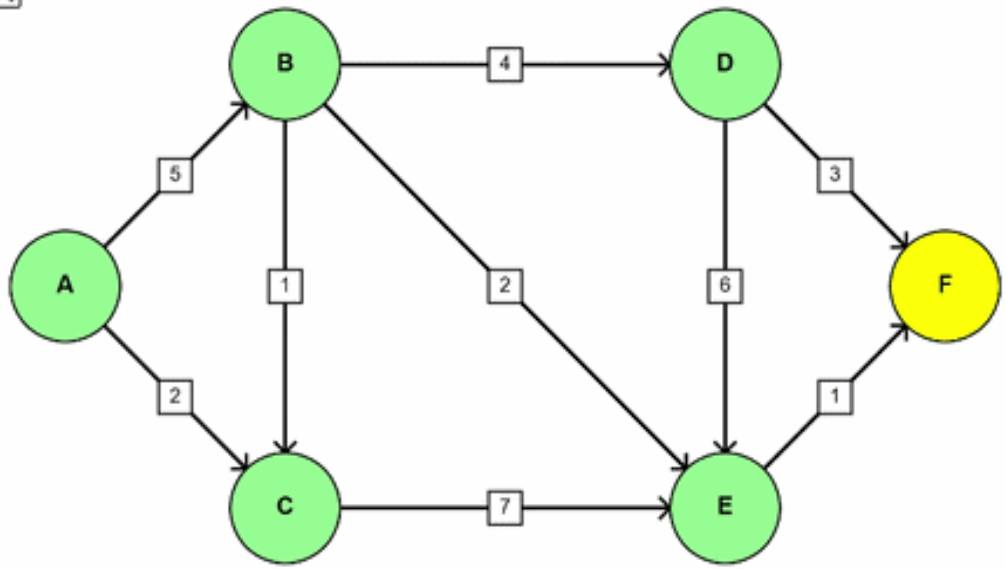
A	C	B	E

D	F					

Shortest Path in Dijkstra

Dijkstra's Algorithm

View on GitHub



Node	Distance	Parent
A	0	
B	5	A
C	2	A
D	9	B
E	7	B
F	8	E

Open

Closed

A	C	B	E	D

F						

Dijkstra's Algorithm

Shortest Path Pseudocode

```
function dijkstra(graph, start):
    visited = [false, false, … , false] # size V
    distances = [∞, ∞, … , ∞] # size V
    distances[start] = 0
    pq = empty priority queue # min-heap
    pq.insert({0, start})
    while pq is not empty:
        index, minValue = pq.poll()
            visited[index] = true
            if distances[index] < minValue: continue
            for (edge in graph[index]):
                if visited[edge.to]: continue
                newDist = distances[index] + edge.cost
                if newDist < distances[edge.to]:
                    distances[edge.to] = newDist
                    pq.insert((edge.to, newDist))
    return distances
```

Dijkstra's Algorithm

Complexity Analysis

Time Complexity

We extract exactly V vertices. Each extraction from a heap takes O(logV)

We evaluate at most E edges. If we relax the edge, we push new value into the heap takes O(log V)

O((V+E)logV)

Space Complexity

We are storing the distances array for every vertex and the priority queue for each relaxed edge

O(V + E)

Dijkstra Optimizing

Lazy Dijkstra: uses a standard Binary Heap. Time complexity: $O((V + E)\log V)$

Eager Dijkstra: uses an indexed priority queue to instantly locate the vertex and update the value dynamically. Time complexity: $O((V + E)\log V)$

D-ary Heap Dijkstra (dense graph optimizer): uses a Heap giving each parent D children, cheaper when updating but more expensive when polling Time complexity: $O(VD\log_D V+E\log_D V)$

Fibonacci Heap Dijkstra: Replaces the standard tree-based heap with a highly complex collection of trees called a Fibonacci Heap which achieved updating time an amortized $O(1)$. Time Complexity: $O(V\log V + E)$

Shortest Path in Bellman-Ford

Bellman-Ford Algorithm: Handling the Negatives

When a graph contains negative weight edges, Dijkstra's greedy logic break down. What should we do?

Bellman-Ford is a **Single Source Shortest Path (SSSP)** algorithm designed specifically to handle graphs with negative weights.

Unlike Dijkstra (Greedy), Bellman-Ford uses **Dynamic Programming**. It calculates shortest paths in a bottom-up manner by gradually allowing paths to use more and more edges.

Bellman & Ford

Richard E. Bellman
(1920–1984)
IEEE Medal of Honor, 1979

Lester R. Ford, Jr.
(1927–)
president of MAA, 1947–48

Shortest Path in Bellman-Ford

How Bellman-Ford works: Global Relaxation

Step 1: Initialization. Set the distance to the source node to *0* and all other nodes to ∞.

Step 2: Edge Relaxation. Iterate through every single edge *(u, v)* with weight *w* in the graph.

If a new shorter path is found, then relax that edge

Step 3: Repeat. Perform this full-graph relaxation exactly ***V - 1* times**.

Why *V – 1* times? *We expect after one loop the path is extended by at least one edge. The shortest path in a graph with V vertices without cycles, contains at least V – 1 edges.*

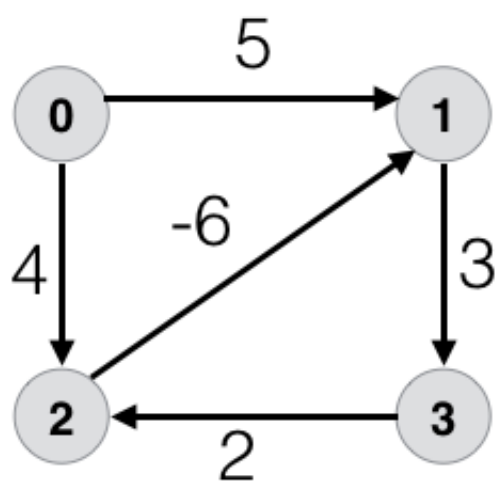
Shortest Path in Bellman-Ford

Bellman-Ford Superpower: Negative Cycle Detection

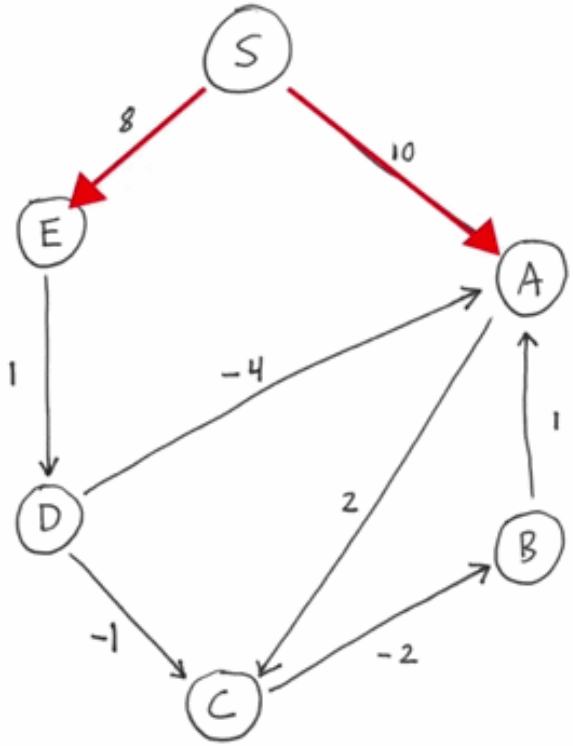
Negative cycle: *A loop in the graph where the sum of the edge weights is less than zero. If one exists, you can loop infinitely to achieve a distance of -∞. No shortest path can be defined.*

After running V – 1 iterations, run the relaxation one more time. If any edge still can be relaxed, that edge is in a negative cycle.

All vertices can be routed from the negative vertices have -∞ distance.



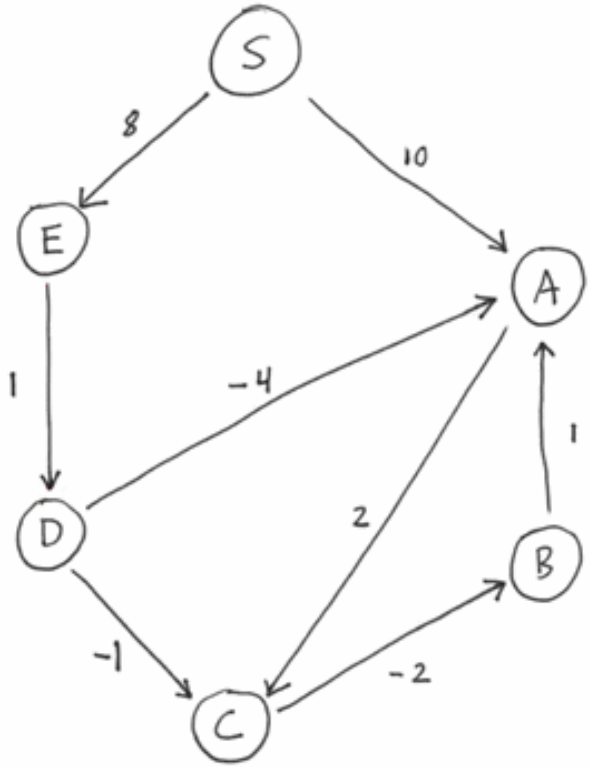
Shortest Path in Bellman-Ford



0	∞	∞	∞	∞	∞
S	A	B	C	D	E

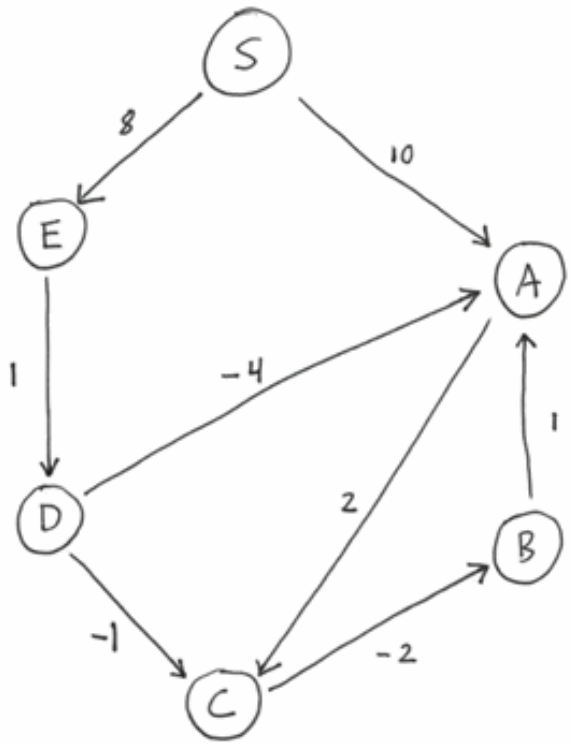
1st Iteration

Shortest Path in Bellman-Ford



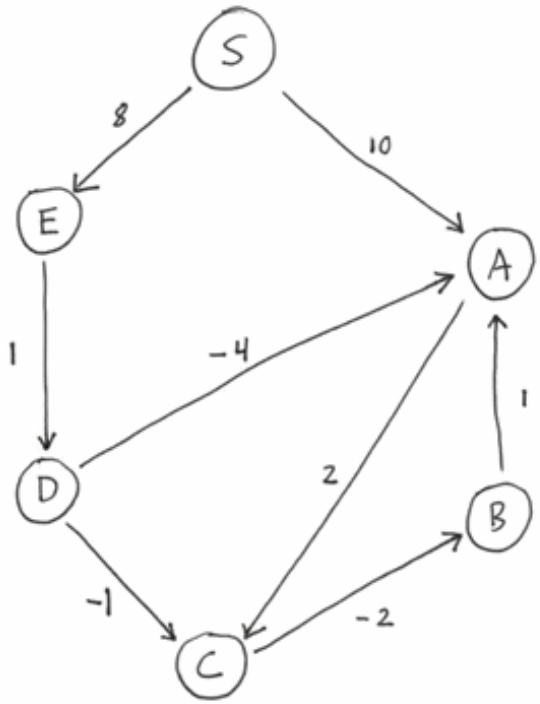
0	10	10	12	9	8
S	A	B	C	D	E

Shortest Path in Bellman-Ford



0	5	10	8	9	8
S	A	B	C	D	E

Shortest Path in Bellman-Ford



0	5	5	7	9	8
S	A	B	C	D	E

Bellman-Ford Algorithm

Bellman-Ford Pseudocode

```
function bellmanford(graph, start):
    dist = [∞, ∞, ... , ∞] # size V
    dist[start] = 0
    for i in range(0, V - 1):
        for edge in graph.edges:
            if dist[edge.from] + edge.cost < dist[edge.to]:
                dist[edge.to] = dist[edge.from] + edge.cost
    negVertices = empty queue
    for edge in graph.edges:
        if dist[edge.from] + edge.cost < dist[edge.to]:"
            distances[edge.to] = -∞
            negVertices.enqueue(edge.to)
    processNegativeCycle(graph, negVertices, dist)
    return dist
```

Bellman-Ford Algorithm

Bellman-Ford Pseudocode

```
function processNegativeCycle(graph, negVertices, dist):
    visited = [false, false, ... , false] # size V
    while negVertices is not empty:
        vertex = negVertices.dequeue();
        if visited[vertex]: continue
        dfs(graph, vertex, negVertices, dist, visited)

function dfs(graph, vertex, negVertices, dist, visited):
    visited[vertex] = true;
    for edge in graph[vertex]:
        if visited[edge.to]: continue
        dist[edge.to] = -∞
        negVertices.enqueue(edge.to)
        dfs(graph, edge.to, negVertices, dist, visited)
```

Bellman-Ford Algorithm

Complexity Analysis

Time Complexity

We iterate every edges V – 1 times and one more loop for negative cycle detection. Therefore, the time complexity is:

O(EV)

Space Complexity

We are storing the distances array for every vertex and the priority queue for each relaxed edge

O(V)

SSSP Algorithm

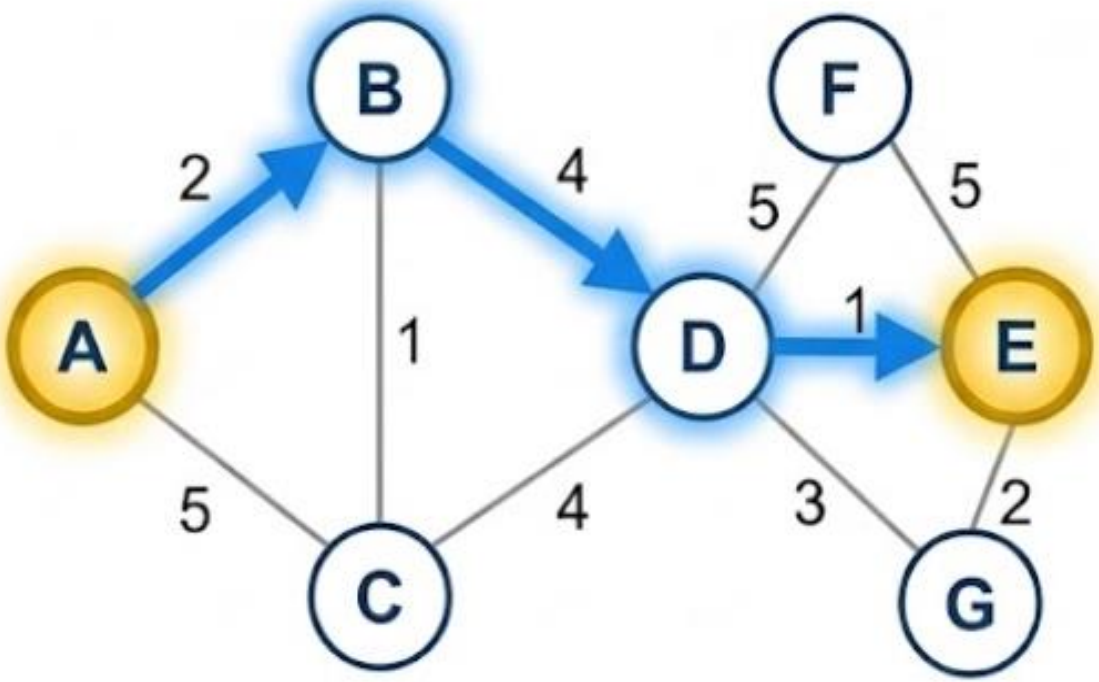
How to track the path?

By introducing a prev array, where prev[i] denotes its previous vertex. We simply update it whenever relaxing the edges

```
if dist[edge.from] + edge.cost < dist[egde.to]:
    dist[edge.to] = dist[edge.from] + edge.cost
    prev[edge.to] = edge.from
```

Then when we need to track the path, we can route from the end to start through the prev array then reversing that result.

All Pairs Shortest Path (APSP)



What is the shortest path between all pairs of nodes?

Floyd-Warshall Algorithm

Floyd-Warshall: APSP

We are no longer looking for paths from a single source. We want to find the shortest path between every pair of vertices *(u, v)* simultaneously

Floyd-Warshall is an APSP dynamic programming algorithm which relies on the idea of comparing between direct connection and routing through an intermediate vertex.

Floyd-Warshall Algorithm

Floyd-Warshall: The Intermediate Node

Can we find a shorter path from i to j by routing it through an intermediate node k?

Initialization: Create a 2D matrix where dist[i][j] is the weight of the direct edge from i to j (or ∞ if no direct edge exists)

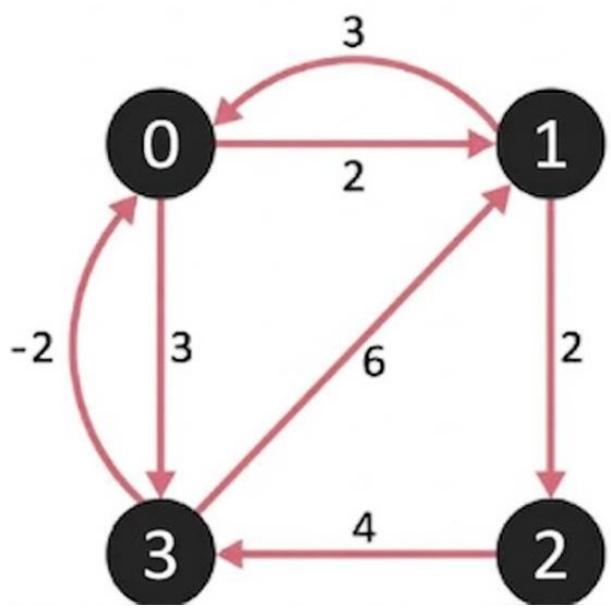
We iterate *V* loops with index of *k,* that dist[i][k] + dist[k][j] is the distance when routing through vertex k. If it is better than the current connection, we update dist[i][j]. We are processing for every pair *(i, j)*

Floyd-Warshall Algorithm Step by Step

First, initialize a 2D distance array:

- Set **dist[i][i] = 0** for all nodes.
- For any edge **(i, j)** with weight **w**, set **dist[i][j] = w**.
- For all other pairs, set **dist[i][j] = +INF**

Graph



Initial Adjacency Matrix

$$\begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & 0 & 2 & \infty & 3 \\ 1 & 3 & 0 & 2 & \infty \\ 2 & \infty & \infty & 0 & 4 \\ 3 & -2 & 6 & \infty & 0 \end{array}$$

Copyright © University of Science, VNUHCM - APCS 2023

Floyd-Warshall Algorithm Step by Step

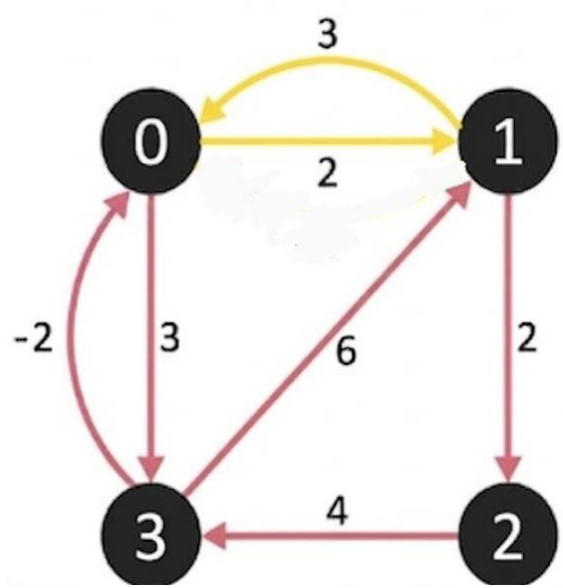
Next, select an intermediate vertex (e.g., Vertex 1):

- Keep the row and column corresponding to Vertex 1 unchanged.
- Iteratively update all other elements in the matrix.
- **Example:** For element `dist[1][1]`, we check if:

```
dist[1][0] + dist[0][1] < dist[1][1]
```

- Since **5 > 0**, no update is required.

Graph



$$\begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & 0 & 2 & \infty & 3 \\ 1 & 3 & 0 & 2 & \infty \\ 2 & \infty & \infty & 0 & 4 \\ 3 & -2 & 6 & \infty & 0 \end{array}$$

Copyright © University of Science, VNUHCM - APCS 2023

Floyd-Warshall Algorithm Step by Step

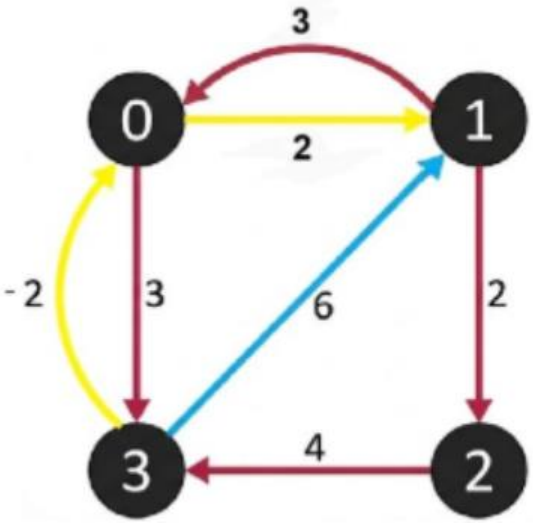
Next, select an intermediate vertex (e.g., Vertex 1):

- Keep the row and column corresponding to Vertex 1 unchanged.
- Iteratively update all other elements in the matrix.
- **Example:** For element `dist[3][1]`, we check if:

```
dist[3][0] + dist[0][1] < dist[3][1]
```

- Since **0<6**, We will update this element.

Graph



Adjacency Matrix

$$\begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & 0 & 2 & \infty & 3 \\ 1 & 3 & 0 & 2 & \infty \\ 2 & \infty & \infty & 0 & 4 \\ 3 & -2 & 6 & \infty & 0 \end{array}$$

Copyright © University of Science, VNUHCM - APCS 2023

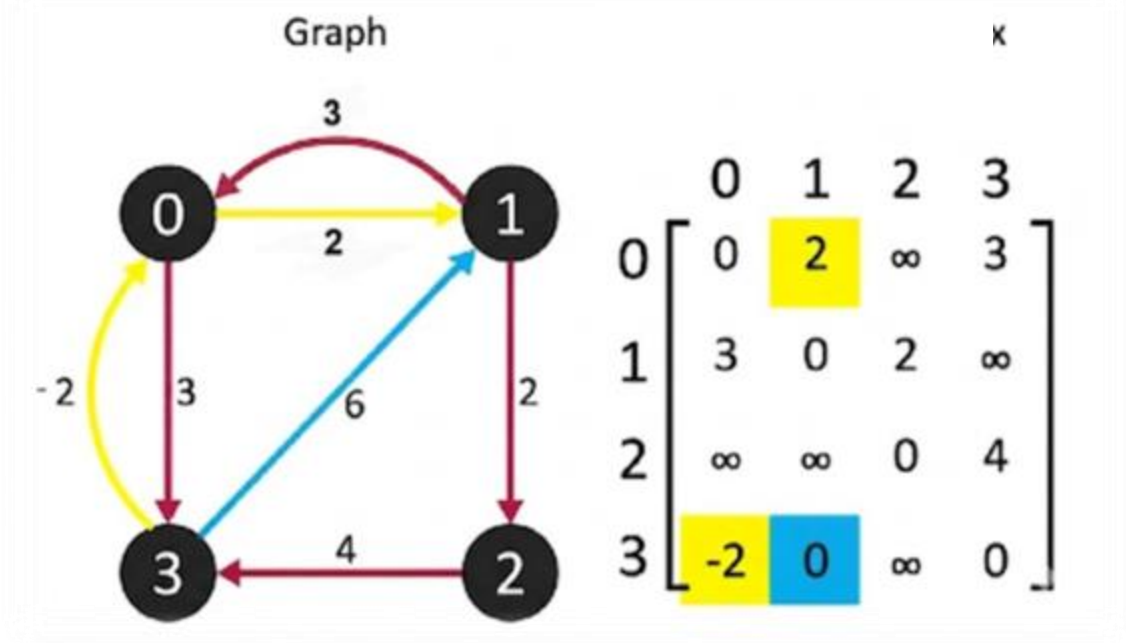
Floyd-Warshall Algorithm Step by Step

Next, select an intermediate vertex (e.g., Vertex 1):

- Keep the row and column corresponding to Vertex 1 unchanged.
- Iteratively update all other elements in the matrix.
- **Example:** For element `dist[3][1]`, we check if:

```
dist[3][0] + dist[0][1] < dist[3][1]
```

- Since **0<6**, We will update this element.



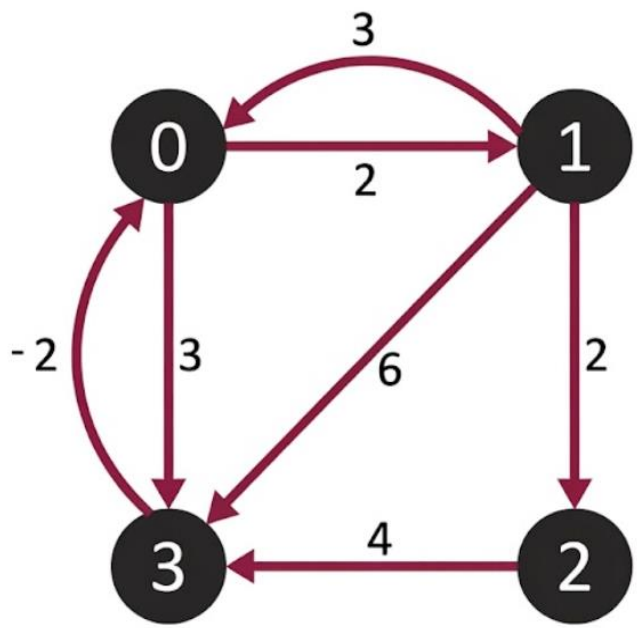
Copyright © University of Science, VNUHCM - APCS 2023

Floyd-Warshall Algorithm Step by Step

After update all other elements in the matrix, we have this matrix.

Iteratively **choosing all other vertex as an intermediate vertex**

Graph



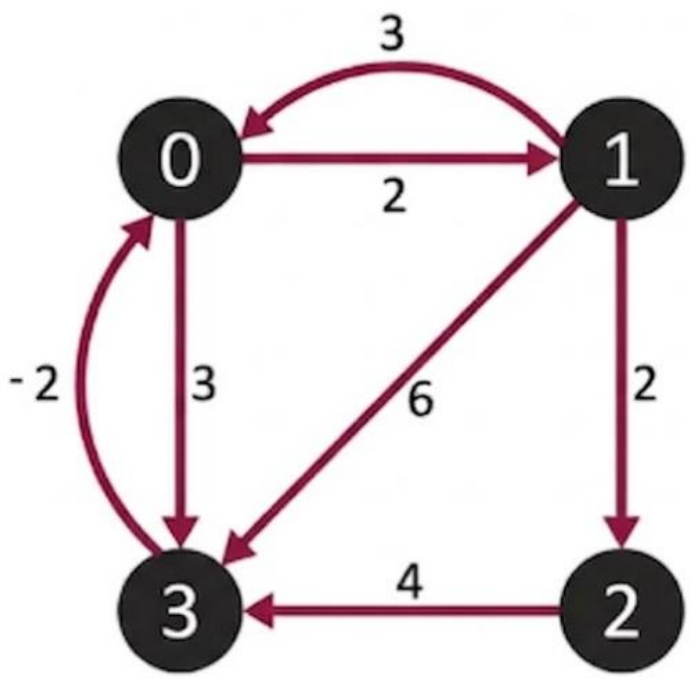
$$
\begin{array}{c|cccc}
 & 0 & 1 & 2 & 3 \\
\hline
0 & 0 & 2 & \infty & 3 \\
1 & 3 & 0 & 2 & 6 \\
2 & \infty & \infty & 0 & 4 \\
3 & -2 & 0 & \infty & 0
\end{array}
$$

Copyright © University of Science, VNUHCM - APCS 2023

Floyd-Warshall Algorithm Step by Step

Finally, we have this matrix with dist[i][j] is the minimum distance from i to j

Graph



Adjacency Matrix

$$\begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & 0 & 2 & 4 & 3 \\ 1 & 3 & 0 & 2 & 6 \\ 2 & 2 & 4 & 0 & 4 \\ 3 & -2 & 0 & 2 & 0 \end{array}$$

Copyright © University of Science, VNUHCM - APCS 2023

Floyd-Warshall Algorithm

Floyd-Warshall Pseudocode

```
function setup(graph, dist):
    for i in range (0, V):
        dist[i][i] = 0
    for edge in graph.edges:
        dist[edge.from][edge.to] = edge.cost

function floydwarshall(graph):
    dist = 2D array of size V x V initialized to ∞
    setup(graph, dist)
    for k in range(0, V):
        for i in range(0, V):
            for j in range (0, V):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

Floyd-Warshall Algorithm

Complexity Analysis

Time Complexity

We iterate V loops to update for every pair of (i, j), leading to the time complexity of:

$$O(V^3)$$

Space Complexity

We only use a 2D matrix to store the distance between every pair of vertices

$$O(V^2)$$

Floyd-Warshall Algorithm

How to track the path?

By introducing a next 2D matrix, where next[i][j] denotes the next node from i when routing to j. We initialize next[i][j] = j if there is a direct connection or next[i][j] = null. After initializing, in the main loop, we update the next matrix as well as the distance

```
if dist[i][k] + dist[k][j] < dist[i][j]:
    dist[i][j] = dist[i][k] + dist[k][j]
    next[i][j] = k
```

Then when we need to track the path, we can route from the start to end through the next matrix.

A* SEARCH ALGORITHM

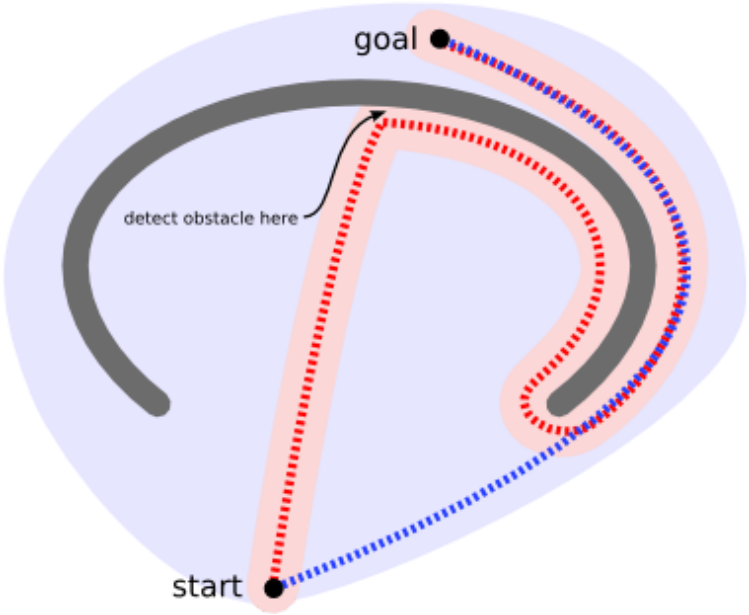
Definition: An efficient and popular search technique used to find the optimal path between two nodes on a graph.

History: Developed in 1968 by Peter Hart, Nils Nilsson, and Bertram Raphael as an extension of Dijkstra's algorithm.

Core Difference:

Dijkstra's Algorithm: Explores blindly in all directions from the starting node.

A-Star (A):* Combines actual movement cost with an estimated future cost (heuristic) to reduce computational overhead and eliminate unnecessary paths.



A* SEARCH ALGORITHM

A*'s search process is guided by the following mathematical formula:

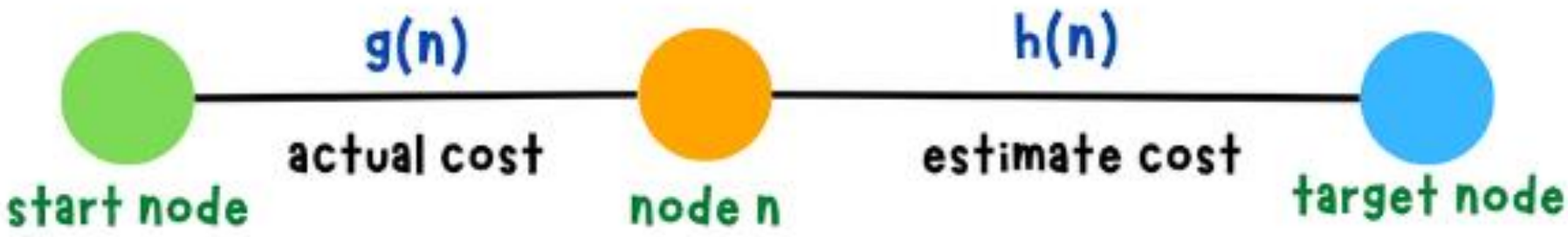
$$f(n) = g(n) + h(n)$$

n: The current node being evaluated on the graph.

g(n): The actual cost to travel from the starting node to node n.

h(n): The Heuristic function – the estimated cost from node n to the target node.

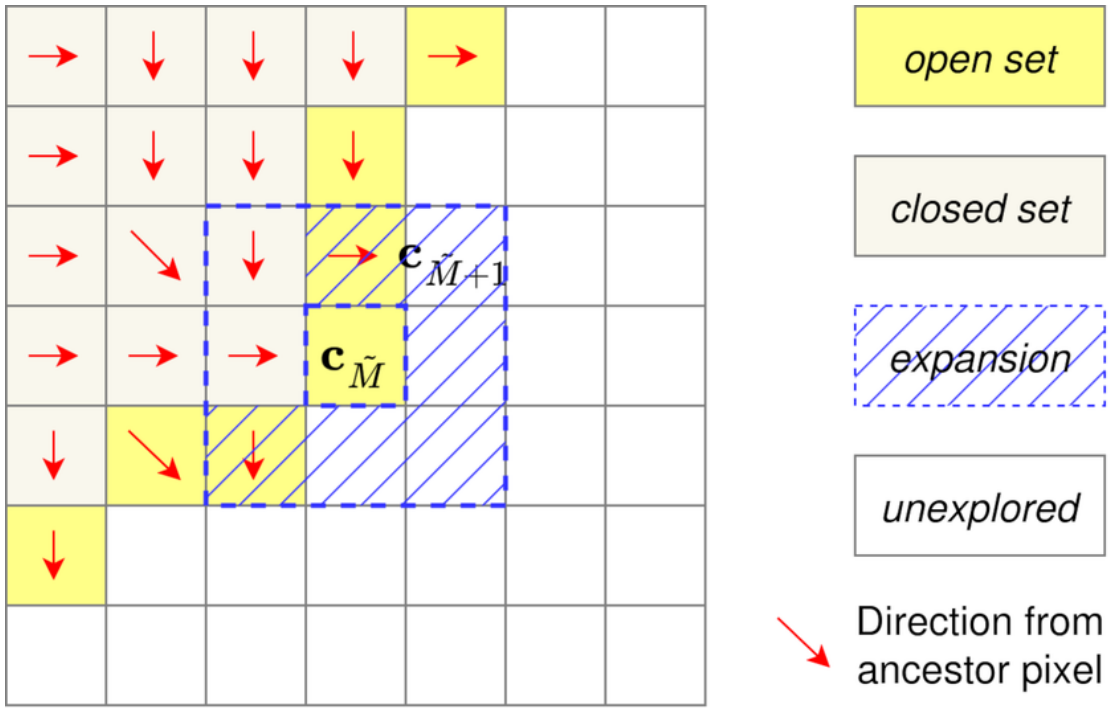
f(n): The lowest estimated total cost for a path going through node n. A* always prioritizes exploring nodes with the lowest f(n) value.



A* SEARCH ALGORITHM

To keep track of where it has been and where it needs to go, A* uses two crucial lists:

- **The Open Set:** Contains nodes that the algorithm has *discovered* (they are neighbors of visited nodes) but has *not yet fully explored*.
- **The Closed Set:** Contains nodes that the algorithm has *already visited and fully evaluated*. The algorithm will never check these nodes again.



A* SEARCH ALGORITHM

Initialization: Calculate f(n) for the starting node and put it into the **Open Set**.

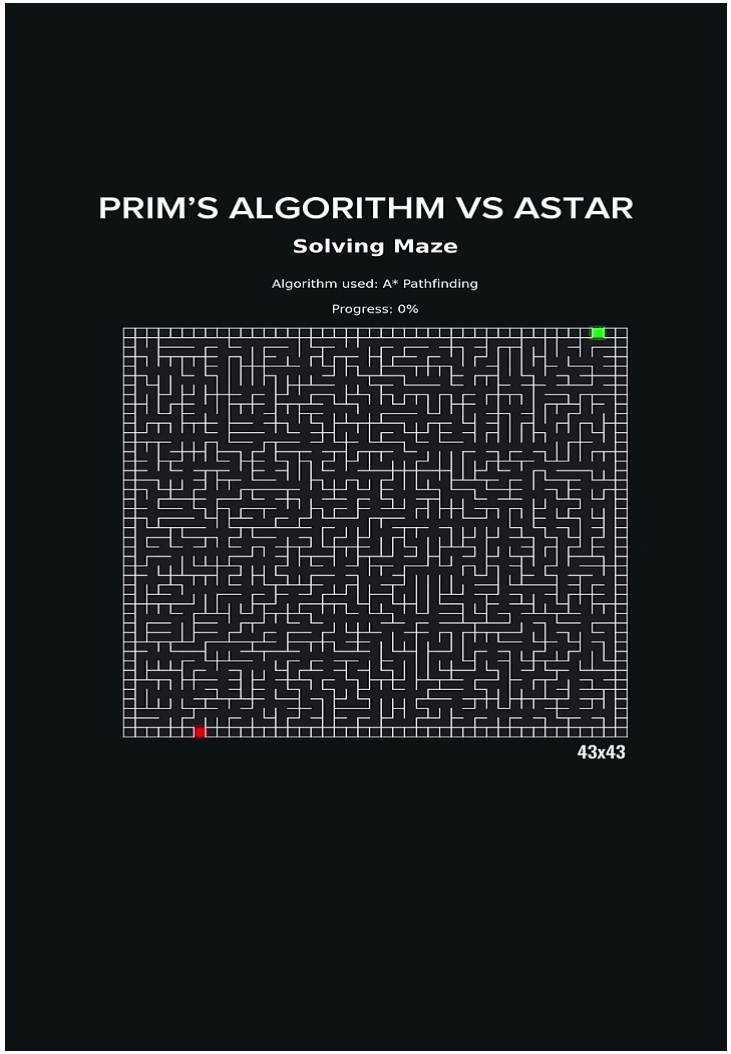
Select the Best Node: Find the node in the **Open Set** with the lowest f(n) value. Remove it from the Open Set and add it to the **Closed Set**.

Explore Neighbors: Look at all adjacent neighbors of the current node.

If a neighbor is in the Closed Set or is an obstacle, ignore it.

Otherwise, calculate its g(n), h(n), and f(n), and add it to the **Open Set**.

Termination: If the current node is the target, the algorithm stops and reconstructs the path. If not, repeat from Step 2.



A* SEARCH ALGORITHM

1. What Makes a Good Heuristic?

•To guarantee finding the shortest path, h(n) must follow two rules:

•**Admissibility:** It must *never overestimate* the actual cost to reach the target.

•**Consistency:** The estimated cost drops logically step-by-step as you get closer to the target, preventing the algorithm from re-evaluating nodes.

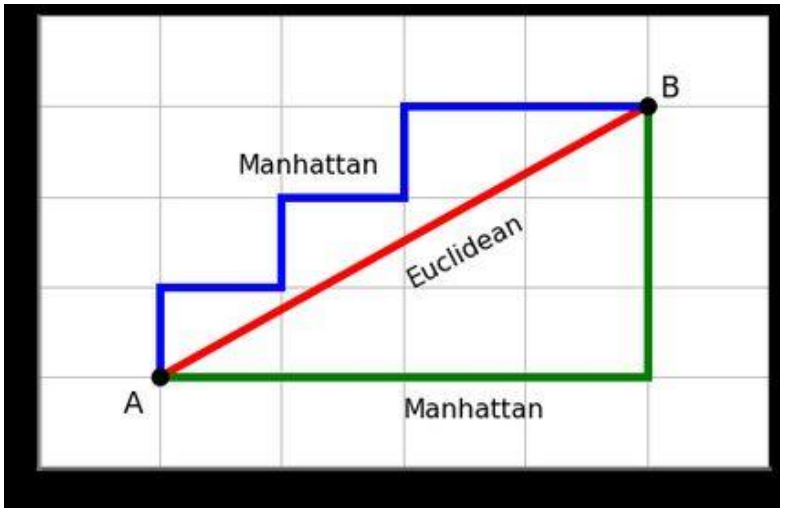
2. Common Heuristic Types:

•**Manhattan Distance:** The sum of absolute differences in X and Y coordinates.

- •*Best for:* Grid-based maps restricted to 4-way movement (Up, Down, Left, Right).

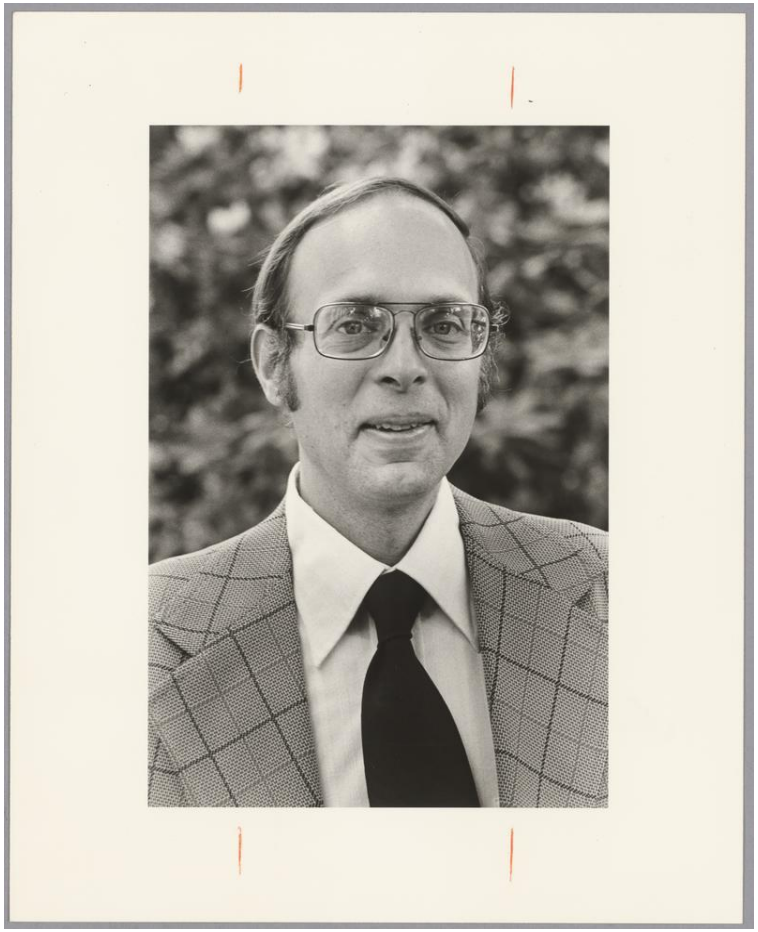
•**Euclidean Distance:** The direct, straight-line distance between two points.

- •*Best for:* Environments where objects can move freely in any direction or angle.



WHAT IS JOHNSON'S ALGORITHM?

- **Definition:** An efficient algorithm for solving the **All-Pairs Shortest Path (APSP)** problem in directed, weighted graphs. It is a hybrid technique that uses **Bellman-Ford** and **Dijkstra** as building blocks.
- **Core Difference:**
- **Dijkstra (Standalone):** Cannot handle graphs with **negative edge weights**.
- **Bellman-Ford (Standalone):** Can handle negative weights, but is too **slow** ($O(V^2E)$) to run for all pairs.
- **Johnson's Algorithm:** Leverages Bellman-Ford for **reweighting** and Dijkstra for **performance**, excelling on sparse graphs with $O(V^2 \log V + VE)$ complexity.



WHAT IS JOHNSON'S ALGORITHM?

Johnson's Solution: Run the lightning-fast Dijkstra's algorithm V times. But first, find a clever way to convert all negative edges into positive ones *without* ruining the shortest paths.

Reweighting:

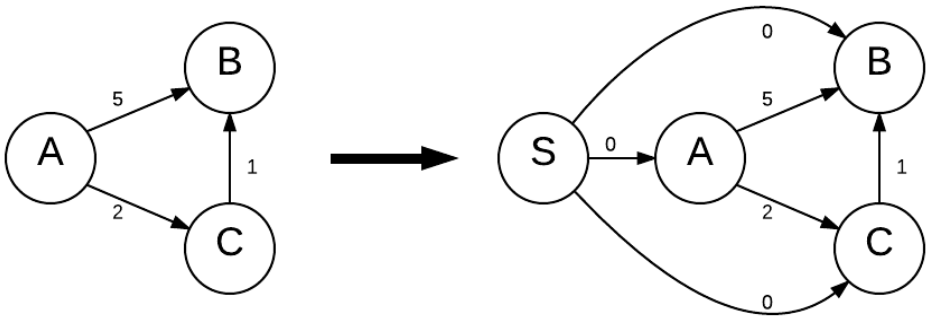
•To achieve this, we recalculate every edge using the differences in vertex "potentials":

•w'(u, v) = w(u, v) + h(u) - h(v)

•**w(u, v):** The original weight of the edge from u to v.

•**h(u), h(v):** The "potential" values of vertices u and v

•**w'(u, v):** The new weight — strictly guaranteed to be larger or equal to 0.



WHAT IS JOHNSON'S ALGORITHM?

1. Add a Dummy Node (S)

Connect S to all other vertices with an edge weight of 0.

2. Run Bellman-Ford (Once)

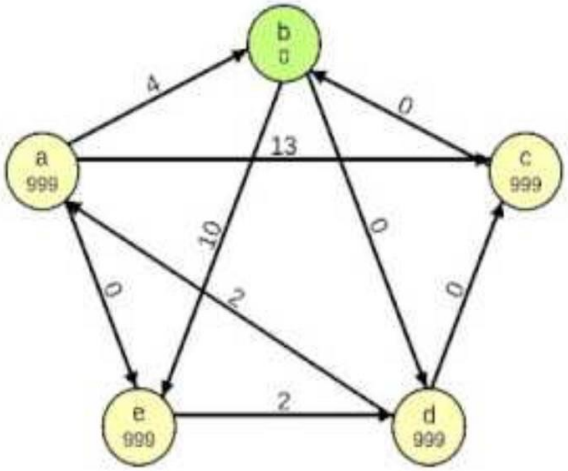
Run it from S to find the potential h(v) for each vertex. *(Halts instantly if a negative cycle is found).*

3. Reweight All Edges

Apply w'(u, v) = w(u, v) + h(u) - h(v) to make all weights larger or equal to 0. Remove S.

4. Run Dijkstra (V times)

Run Dijkstra from every vertex. Finally, reverse the formula to get the original distances.



Choosing minimum-cost vertex

Multi Source Shortest Path (MSSP)

Multi Source Shortest Path (MSSP): The Super-Source Trick

The Problem: *Instead of one starting point, what if we have a set of multiple starting nodes?*

e.g., "Find the distance to the nearest hospital for every house in the city," where there are multiple hospitals

We do not need to invent a new algorithm or run Dijkstra multiple times. We can reduce MSSP back down to a standard Single-Source problem using the **"Super-Source"** trick.

1. Create a single dummy (super source)
2. Draw a directed edge from this super source to every actual starting node with a weight of 0.
3. Run a standard SSSP (Dijkstra, Bellman-Ford, etc) starting from the dummy super source

C++ Libraries for Shortest Path

1. Boost Graph Library (BGL)
2. LEMON (Library for Efficient Modeling and Optimization in Networks)
3. Google OR-Tools (Graph Module)

Real-life Performance

Sources:

- Stanford Large Network Dataset Collection – STAN
  - ego-Twitter
  - ego-Gplus
  - ego-Facebook
- 9th DIMACS Implementation Challenge (with coordinates)
  - New York City

Copyright © University of Science, VNUHCM - APCS 2023

Algorithm Efficiency (Review)

Algorithm	*Time Complexity*	*Space Complexity*	*Best Use Case*
Dijkstra	O(E * logV)	O(V)	Single-source, Non-negative weights
A Search*	O(E) (Heuristic dependent)	O(V)	Goal-oriented navigation
Bellman-Ford	O(V * E)	O(V)	Graphs with negative weights
Floyd-Warshall	O(V^3)	O(V^2)	All-pairs shortest path

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Dataset: Facebook Ego-network

- **Nodes:** 4,039
- **Edges:** 88,234
- **Type:** Undirected Graph

Constraints:

- **Time Limit:** 60.0 Seconds
- **Memory Limit:** 2.0 GB

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Algorithm	*Time (ms)*	*Memory (MB)*	*Status*
Dijkstra	39.87 ms	0.3652 MB	SUCCESS
A Search*	94.40 ms	0.5102 MB	SUCCESS
Bellman-Ford	34.78 ms	0.2712 MB	SUCCESS
Floyd-Warshall	N/A	CRASH	MEMORY ERROR

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Dataset: Twitter Ego-network

- **Nodes:** 81,306
- **Edges:** 1,768,149
- **Type:** Directed Graph

Constraints:

- **Time Limit:** 60.0 Seconds
- **Memory Limit:** 2.0 GB

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Algorithm	*Time (ms)*	*Memory (MB)*	*Status*
Dijkstra	87.56 ms	4.7501 MB	SUCCESS
A Search*	79.52 ms	4.7480 MB	SUCCESS
Bellman-Ford	1515.49 ms	4.7478 MB	SUCCESS
Floyd-Warshall	N/A	CRASH	MEMORY ERROR

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Dataset: GPlus Ego-network

- **Nodes:** 107,614
- **Edges:** 30,494,866
- **Type:** Directed Graph

Constraints:

- **Time Limit:** 60.0 Seconds
- **Memory Limit:** 2.0 GB

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Algorithm	*Time (ms)*	*Memory (MB)*	*Status*
Dijkstra	16286.19 ms	16.7066 MB	SUCCESS
A Search*	10695.59 ms	29.0760 MB	SUCCESS
Bellman-Ford	TIMEOUT	---	FAILED
Floyd-Warshall	N/A	CRASH	MEMORY ERROR

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset

Dataset: New York City Road Network

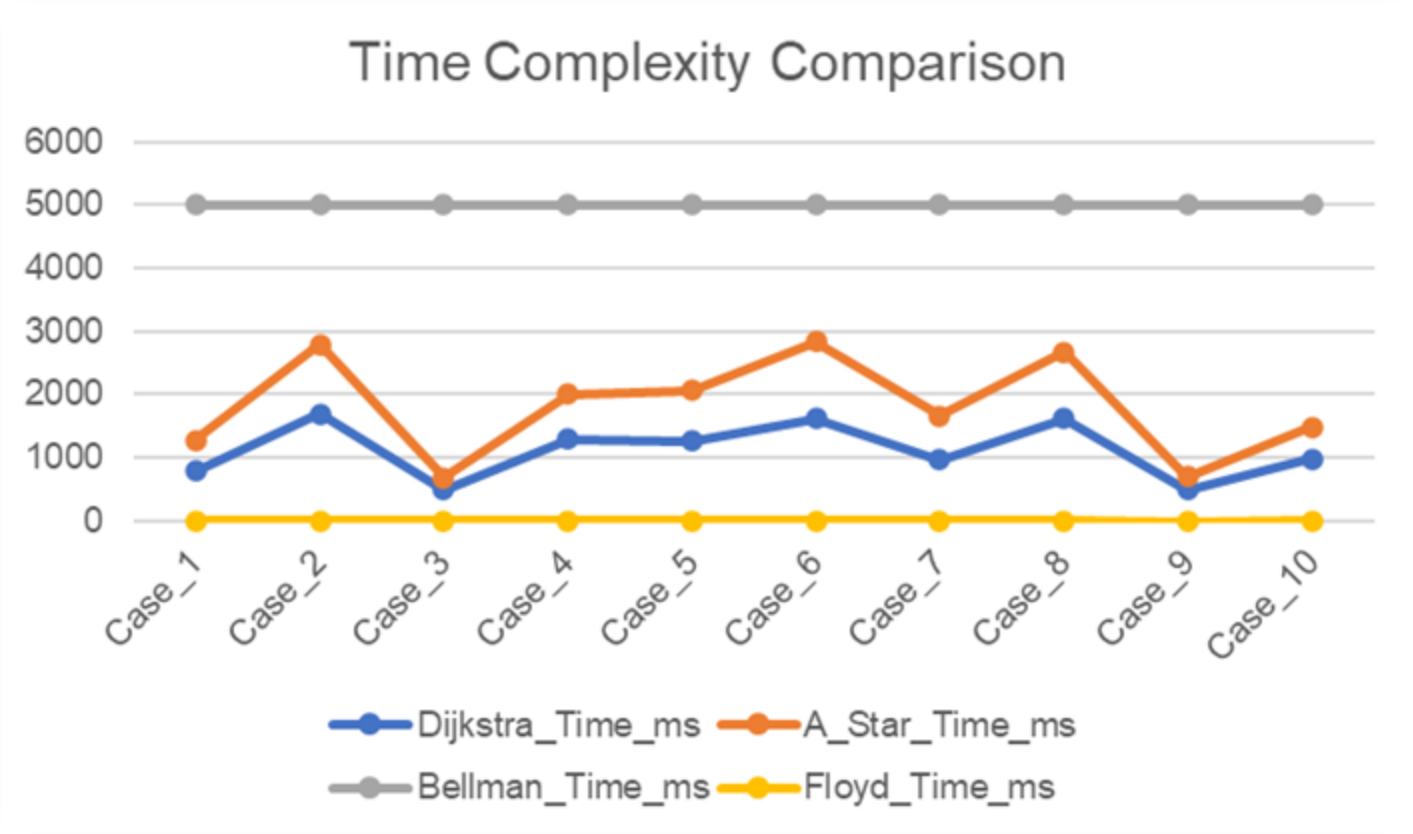
- **Nodes:** 264,346
- **Edges:** 733,846
- **Coordinates:** Geographic Co.

Constraints:

- **Time Limit:** 5.0 Seconds
- **Memory Limit:** 2.0 GB

Copyright © University of Science, VNUHCM - APCS 2023

Real-life Dataset



Notes: Floyd exceeded memory constraint so we set its TC to 0

Copyright © University of Science, VNUHCM - APCS 2023

Test Conclusion

For Large-Scale Networks (Twitter/Facebook):

- **Dijkstra** is the industry standard for efficiency.
- **A*** is superior when geographical/heuristic data is available.

Avoiding:

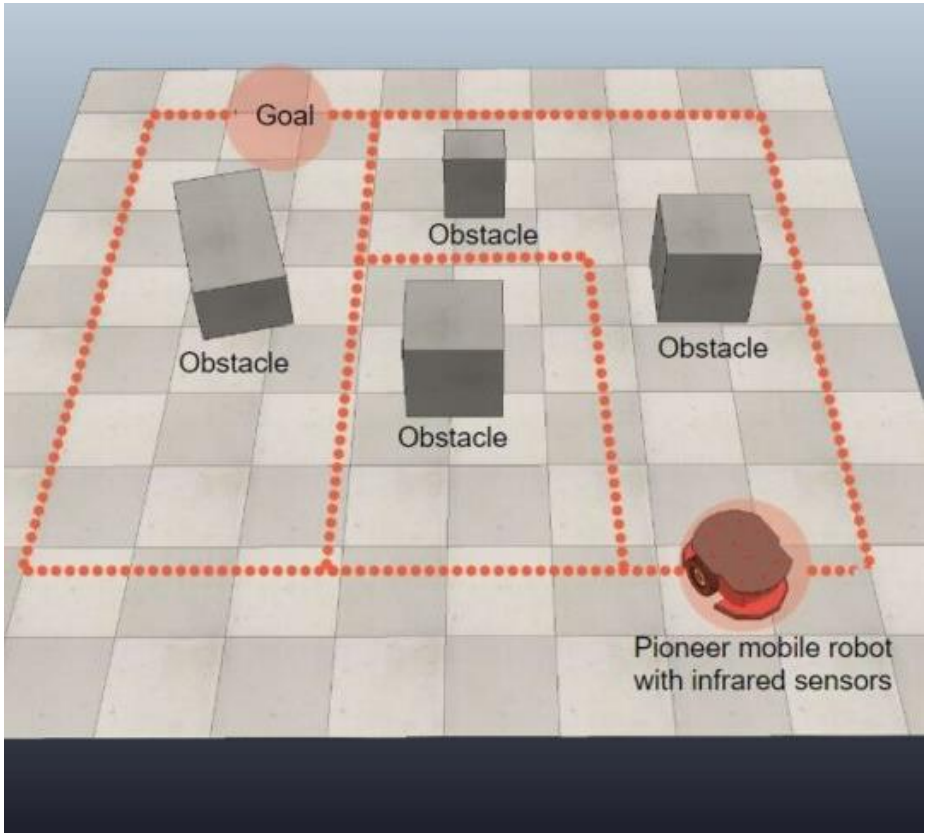
- **Bellman-Ford** unless negative edges are present.
- **Floyd-Warshall** for any graph exceeding 10,000 nodes.

Copyright © University of Science, VNUHCM - APCS 2023

Applications

Real-World Applications of Shortest Path Algorithms

- **Modern Logistics Systems:** Optimizing delivery routes and supply chain distribution.
- **Telecommunications & Postal Networks:** Routing data packets and mail efficiently across complex networks.
- **Automation & Artificial Intelligence:** Path finding for robotics and NPC (Non-Player Character) navigation.



Copyright © University of Science, VNUHCM - APCS 2023

Applications

Single Source Shortest Path (SSSP)

- **Topological Sort (DAGs)**: Project Scheduling (PERT/CPM); Dependency Resolution.
- **Bellman-Ford**: Arbitrage Trading in Forex markets; Distance-Vector Routing protocols (e.g., RIP for small networks).
- **Lazy / Eager Dijkstra**: Link-State Routing protocols (e.g., OSPF in Internet backbones); General logistics systems.

Single-Pair Shortest Path (SPSP)

- **Breadth-First Search (BFS)**: Pathfinding in unweighted graphs (e.g., Social network degrees of separation).
- **A***: GPS Navigation; 2D/3D Game Character Routing; Robotic Automation pathfinding.

Copyright © University of Science, VNUHCM - APCS 2023

Applications

All-Pairs Shortest Path (APSP)

- **Floyd-Warshall**: Pre-computing Static Distance Matrices; Transitive Closure calculation; City traffic network optimization.
- **Johnson's Algorithm:** Identifying shortest paths between all node pairs in sparse graphs containing negative edge weights.

Multi-Source Shortest Path (MSSP)

- **Super Source Technique:** Optimizing the nearest facility location (e.g., dispatching emergency services).

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

1. In the **A* search algorithm's** evaluation function **f(n) = g(n) + h(n)**, what exactly does **h(n)** represent?

A) The exact accumulated cost from the start node to node **n**.

B) The total number of edges traversed so far.

C) The estimated cost from node **n** to the destination.

D) The time complexity of evaluating the current node.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

1. In the **A* search algorithm's** evaluation function **f(n) = g(n) + h(n)**, what exactly does **h(n)** represent?

A) The exact accumulated cost from the start node to node **n**.

B) The total number of edges traversed so far.

C) The estimated cost from node n to the destination.

D) The time complexity of evaluating the current node.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

2. Why does **Dijkstra's algorithm** fail to find the correct shortest path in graphs with negative edge weights?

A) The priority queue data structure cannot process negative integer values.

B) It greedily marks nodes as "visited" under the assumption that path costs can never decrease.

C) Negative edges immediately crash the algorithm due to division by zero errors.

D) It relies on a heuristic function that strictly requires positive inputs.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

2. Why does **Dijkstra's algorithm** fail to find the correct shortest path in graphs with negative edge weights?

A) The priority queue data structure cannot process negative integer values.

B) It greedily marks nodes as "visited" under the assumption that path costs can never decrease.

C) Negative edges immediately crash the algorithm due to division by zero errors.

D) It relies on a heuristic function that strictly requires positive inputs.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

3. What are the standard time and space complexities of the **Floyd-Warshall algorithm**, respectively (where **V** is the number of vertices and **E** is the number of edges)?

A) Time: **O(V^2)**, Space: **O(V^3)**

B) Time: **O(V^3)**, Space: **O(V^2)**

C) Time: **O(E *log V)**, Space: **O(V)**

D) Time: **O(V^2 log E)**, Space: **O(V^2)**

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

3. What are the standard time and space complexities of the **Floyd-Warshall algorithm**, respectively (where **V** is the number of vertices and **E** is the number of edges)?

A) Time: **O(V^2)**, Space: **O(V^3)**

B) Time: O(V^3), Space: O(V^2)

C) Time: **O(E *log V)**, Space: **O(V)**

D) Time: **O(V^2 log E)**, Space: **O(V^2)**

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

4. A graph has negative edges. To use Dijkstra, a student adds a constant **C** to **ALL** edges to make them larger or equal to 0, runs Dijkstra, and subtracts the accumulated constant later. Is this method correct or incorrect, and why?

A. Excellent, absolutely correct. This is a classic graph transformation technique that leverages Dijkstra's speed without distorting the final result.

B. Incorrect. Altering the weights by adding a constant will introduce hidden negative cycles into the graph, causing Dijkstra's algorithm to fall into an infinite loop.

C. Acceptable, but the graph must be strictly undirected. If it is a directed graph, adding a constant will reverse the traversal direction of Dijkstra's algorithm.

D. A fundamental flaw. Adding a constant C to every edge heavily penalizes paths made up of a larger number of edges. A path with many edges will have **C** added to it multiple times compared to a path with fewer edges, completely invalidating the shortest path calculation.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

4. A graph has negative edges. To use Dijkstra, a student adds a constant **C** to **ALL** edges to make them larger or equal to 0, runs Dijkstra, and subtracts the accumulated constant later. Is this method correct or incorrect, and why?

A. Excellent, absolutely correct. This is a classic graph transformation technique that leverages Dijkstra's speed without distorting the final result.

B. Incorrect. Altering the weights by adding a constant will introduce hidden negative cycles into the graph, causing Dijkstra's algorithm to fall into an infinite loop.

C. Acceptable, but the graph must be strictly undirected. If it is a directed graph, adding a constant will reverse the traversal direction of Dijkstra's algorithm.

D. A fundamental flaw. Adding a constant C to every edge heavily penalizes paths made up of a larger number of edges. A path with many edges will have C added to it multiple times compared to a path with fewer edges, completely invalidating the shortest path calculation.

=>Correct! Suppose Path 1 consists of 1 edge (weight 10), and Path 2 consists of 3 edges (total weight 6). Originally, Path 2 is the shortest. If we add C = 5 to all edges: Path 1 becomes 15, and Path 2 becomes 6 + (3 * 5) = 21. Suddenly, Path 1 appears shorter. Dijkstra will return the wrong path

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

5. You manage a regional delivery network. A customer requests a pickup. You need to find the most efficient route from *any* of your multiple local branches to this single customer's location to dispatch a driver.
Which category of shortest-path problem are you solving?

A) Single-Source Shortest Path (SSSP)

B) Single-Pair Shortest Path (SPSP)

C) All-Pairs Shortest Path (APSP)

D) Multi-Source Shortest Path (MSSP)

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

5. You manage a regional delivery network. A customer requests a pickup. You need to find the most efficient route from *any* of your multiple local branches to this single customer's location to dispatch a driver. **Which category of shortest-path problem are you solving?**

A) Single-Source Shortest Path (SSSP)

B) Single-Pair Shortest Path (SPSP)

C) All-Pairs Shortest Path (APSP)

D) Multi-Source Shortest Path (MSSP)

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

6. Assume the current shortest path from Node A to Node B uses 5 edges, while an alternative path uses only 2 edges (but has a slightly higher total weight). If a new toll tax of +$10 is added to *every single edge* in the graph, **what might happen?**

A) The shortest path is guaranteed to remain exactly the same.

B) The algorithm will crash because the graph's topology was altered.

C) The alternative path (with 2 edges) might become the new shortest path.

D) The total cost of all paths will increase by exactly $10.

Copyright © University of Science, VNUHCM - APCS 2023

QUIZ

6. Assume the current shortest path from Node A to Node B uses 5 edges, while an alternative path uses only 2 edges (but has a slightly higher total weight). If a new toll tax of +$10 is added to *every single edge* in the graph, **what might happen?**

A) The shortest path is guaranteed to remain exactly the same.

B) The algorithm will crash because the graph's topology was altered.

C) The alternative path (with 2 edges) might become the new shortest path.

D) The total cost of all paths will increase by exactly $10.

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

1. Communication Service (VNOJ)

Link: Communication Service - VNOJ: VNOI Online Judge

Network Setup: A directed graph containing **n** nodes and **m** one-way edges.

- Each edge from node **u** to node **v** has a specific transmission delay **c(u, v)**.

Initial State: All **n** nodes start in an **inactive** state (they cannot forward or route any messages).

Dynamic Operations (k queries):

- **Type 1 (Activate):** Given node **x**, switch it to active mode. It can now be used as an intermediate router.
- **Type 2 (Query):** Given nodes **x** and **y**, find the shortest path (minimum total delay) from **x** to **y**.

Core Constraint: For any Type 2 query, the path can **ONLY** pass through intermediate nodes that are currently active.

Required Output: Print the minimum delay for every Type 2 query. If no valid path exists, output `-1`.

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

1. Communication Service (VNOJ)

Link: Communication Service - VNOJ: VNOI Online Judge

Instructions to solving problem:

Core Idea: Dynamic implementation of the **Floyd-Warshall algorithm.**

Editorial Strategy:

- Convert the graph into an Adjacency Matrix.
- Initialize edges without direct connections to +infinity, except for the main diagonal.
- When a node **x** is activated for transmission, use **x** as the intermediate node to update the entire matrix.

For delay query operations, directly return the corresponding matrix value (if the value remains +infinity, return -1).

Complexity Analysis:

- **Time complexity:** O(N^2) for query 1, O(1) for query 2
- **Space complexity:** O(N^2) for Adjacency matrix

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

2. Flight Discount (CSES)

Link: CSES - Flight Discount

Network Setup: A directed graph containing **n** cities (nodes) and **m** unidirectional flights (edges). Each flight from city **a** to city **b** has a specific price **c**.

Initial State: You start at City 1 (Syrjälä) and need to navigate to City **n** (Metsälä). You are holding exactly **one discount coupon**.

Core Constraint (The Coupon): You can use the coupon to halve the price of any **single** flight during your route. If used on a flight with price **x**, the cost becomes **x/2** (rounded down). The coupon can only be used once.

Required Output: Print a single integer representing the absolute minimum total price to travel from City 1 to City **n**.

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

2. Flight Discount (CSES)

Link: CSES - Flight Discount

Core Idea: Dijkstra's Algorithm on a State-Space Graph.

Editorial Strategy:

- Create a 2D distance array price size n*2. price[i][0] stores the minimum cost to reach node **i** without utilizing the discount coupon, while price[i][1] stores the minimum cost after utilizing the coupon.
- During state extraction from the Min-Heap: If the extracted state has already used the coupon, only update adjacent paths assuming the coupon is used. If the coupon has not been used, branch out to update both states (using the coupon on the current edge vs. saving it for later).

Complexity Analysis:

- **Time complexity:** O(n*logm)
- **Space complexity:** O(n)

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

3. Shortest Path in a Binary Matrix

Link: Shortest Path in Binary Matrix- Leetcode

Network Setup: An **n*n** binary matrix (`grid`) representing a map. A value of `0` indicates a clear, walkable cell, while `1` indicates a blocked cell (obstacle). Adjacent cells are connected **8-directionally** (horizontally, vertically, and diagonally).

Initial State: You start the traversal at the top-left cell `(0, 0)`.

Core Constraint: You can only move through clear cells (`0`). The destination is the bottom-right cell `(n - 1, n - 1)`.

Required Output: Return the length of the shortest clear path. The length is defined as the total number of visited cells on the path. If no valid path exists, return `-1`

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

3. Shortest Path in a Binary Matrix

Link: Shortest Path in Binary Matrix- Leetcode

Core Idea: A* Algorithm on a Grid.

Editorial Strategy:

- Use the heuristic function h(n) = min(dx, dy) + |dx - dy| to optimize the search space toward the destination.

Complexity Analysis:

- **Time complexity:** O(N^2)
- **Space complexity:** O(N^2)

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

4. Nearest Fire Station

1. **Problem Statement:** You are developing a dispatch system for a city's Fire Department. The city has **N** intersections and **M** roads. There are **K** fire stations located at distinct intersections. When a fire is reported at any intersection, the system must automatically dispatch an engine from the station with the shortest travel time.

2. **Problem Analysis:**

Network Setup: An undirected, weighted graph. Nodes = Intersections. Edges = Roads. Edge weights = Travel time.

Initial State: Multiple source nodes (**K** fire stations) and multiple queries (**Q** fire locations).

Core Constraint: Dispatch the station with the *absolute shortest travel time*. If multiple stations share the exact same minimum travel time, return any stations.

Required Output: For each query, print the dispatched station's ID and the travel time.

Copyright © University of Science, VNUHCM - APCS 2023

PROGRAMMING ASSIGNMENTS

4. Nearest Fire Station

Problem type: Multi-Source Shortest Path (MSSP).

Core Idea: (Super Source Simulation) Introduce a virtual "Super Source" node with **0-weight** edges connecting to all **K** fire stations to simulate a Single-Source Shortest Path (SSSP) traversal.

Path Tracking: Utilize an array to track the origin paths, allowing the algorithm to trace back and identify the exact originating fire station for every node.

Complexity Analysis:

- **Time complexity: O(MlogN)**
- **Space complexity: O(N+M)**

Copyright © University of Science, VNUHCM - APCS 2023

THANHS FOUR WASHING

Copyright © University of Science, VNUHCM - APCS 2023